PROO 3 - Assignment: Medical Practice

Objective

Create a waiting room management tool for a medical practice.

Things To Learn

- Applying the *Model-View-Controller*-pattern utilizing the *Observer*-pattern.
- Making first contact with `FunctionalInterface`s and *lambda expressions*.

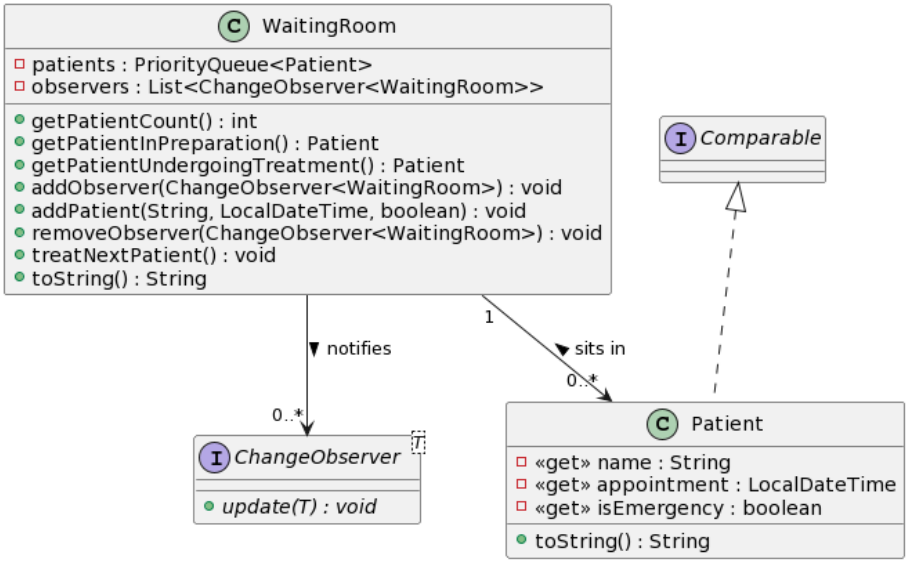
Submission Guidelines

- Your implemented solution as ***zipped*** *IntelliJ*-project.

Task

Your local doctor's practice has tasked you with creating an application for registering and queueing their patients. Apply the *Model-View-Controller*-pattern for a clean solution:

Model



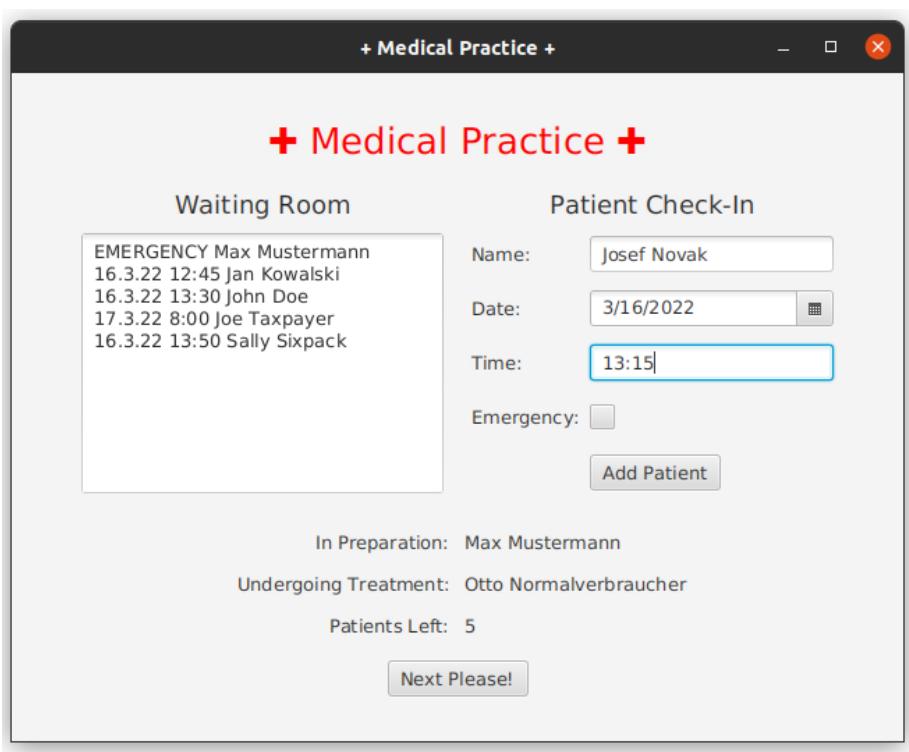
The *model*-part consists of two classes:

1. `Patient`

    1. Each patient has a name, an appointment and a *flag* that determines, whether it's an emergency or not.
    2. Patients should be *comparable*, so that critical cases come before regular patients. Two patients having the same *emergency status* are ordered by their appointment date and time.
    3. You can use the following `DateTimeFormatter` in combination with the appointment's `format`-method for the `String` representation of a patient: `DateTimeFormatter.ofPattern("dd.M.yy H:mm")`
2. `WaitingRoom`
    1. The waiting room stores the patients using a `PriorityQueue`. Calling the `treatNextPatient`-method removes the patient next in line and sends him/her *into treatment*.
    2. Additionally, a list of *observers* is stored. They are notified, when any *changes* occur.

Luckily (and also slightly disturbing), the doctor enjoys writing *unit-tests* in his spare time and provided them along with this assignment. Make sure everything works before continuing to the next parts.

View



The following *controls* are

probably new to you:

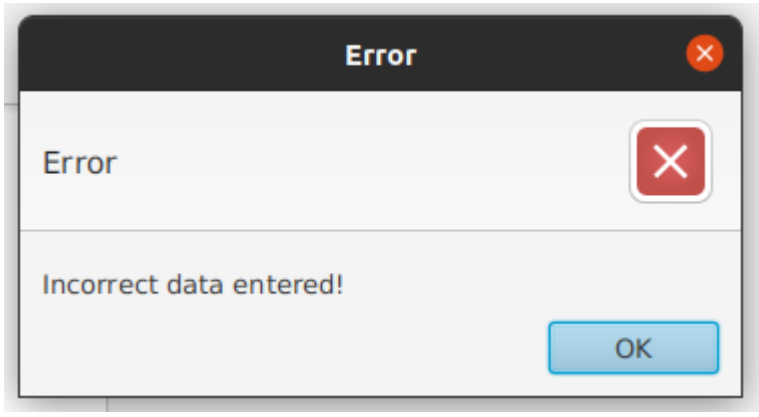
- The patients in the waiting room are displayed using a `TextArea`. Make sure that the control is **not *editable.***
- A `DatePicker` is used for the appointment's date.
- The emergency status can be set using a `Checkbox`.

As always, don't forget to set `fx:id`s and *event handlers* where necessary!

Controller

Time to put it all together! Check the provided *GIF* and the following remarks for implementation details:

- Use a `ChangeObserver<WaitingRoom>` to register to changes in the model. Perform all updates on the view in the implementation of the `FunctionalInterface`!
  - You may - of course - use a *lambda expression*.
- Note, that iterating the elements of a `PriorityQueue` **does not reflect their order** - hence, this is not a requirement for the display, as long as the patients are treated in the correct order.
- The `DatePicker` should be set to today's date by default. You can access the `LocalDate` object using `getValue` and `setValue`.
- As soon as a patient is (successfully) added, the elements of the *check-in*-form should be reset.
- Checking the *emergency*-checkbox should automatically set the appointment's date and time to *now*, so the form can be filled out quicker.
  - The `onAction`-event is triggered, when a checkbox gets checked or unchecked.
  - `isSelected` can be used to check whether the checkbox is checked. You can check or uncheck a checkbox using `setSelected`.
- The *Next Please!*-button should be `disabled`, when the waiting room is empty.
- An `Alert` should be displayed, when the data entered on check-in is erroneous or incomplete (all fields need to be filled out!):


Error
Error
Incorrect data entered!
OK